

راهنمای کاربردی دیزاین پترن‌های پرکاربرد در جاوا

دیزاین پترن یا الگوی طراحی، یک راه حل کلی و قابل تکرار برای حل یک مشکل رایج در طراحی نرمافزار است، که به جای کدنویسی از صفر، یک قالب یا نقشهٔ اثبات شده برای ساختاردهی کد و بهبود انعطافپذیری و نگهداری آن در اختیار برنامه‌نویس قرار میدهد.

ساختار:

• برای هر الگو:

• مشکل رایج (نشانه استفاده)

• راه حل (ایده پترن)

• مثال ساده در جاوا

• مزایا

• معایب / هشدارها

۱. Creational (ایجاد)

این الگوها به نحوه ساخته شدن اشیاء می‌پردازند. کارشان این است که فرایند `new` کردن را هوشمندانه‌تر کنند تا سیستم از جزئیات ساخت شیء بی‌خبر باشد و کد انعطاف‌پذیرتر شود. مثال معروف: Singleton (تضمین یک نمونه منحصر به فرد)، Factory (ساخت اشیاء بدون مشخص کردن کلاس دقیق آنها).

۲. Structural (ساختاری)

این الگوها به ترکیب کلاس‌ها و اشیاء برای ساختن ساختارهای بزرگ‌تر می‌پردازند. هدفشان این است که با استفاده از ارث‌بری یا ترکیب، رابط‌های ساده‌تری برای کار با سیستم‌های پیچیده فراهم کنند. مثال معروف: Adapter (هماهنگ‌کننده دو رابط ناسازگار)، Decorator (افزودن قابلیت جدید به اشیاء به صورت پویا).

۳. Behavioral (رفتاری)

این الگوها به ارتباط و تعامل بین اشیاء می‌پردازند. کارشان تخصیص مسئولیت‌ها بین اشیاء و مدیریت الگوریتم‌ها و جریان داده است تا ارتباطات به هم گره نخورده و قابل توسعه باشند. مثال معروف: Observer (اعلام تغییرات به اشیاء وابسته)، Strategy (تغییر الگوریتم در زمان اجرا).

۴. Concurrency (همروندی / همزمانی)

این الگوها به مدیریت همزمانی و چندنخی (Multi-threading) می‌پردازند. کارشان این است که به اشیاء کمک کنند در محیط‌های همروند (که چندین کار با هم اجرا می‌شوند) بدون بروز مشکل کار کنند و از تداخل نخ‌ها جلوگیری شود. مثال معروف: Active Object (جداسازی اجرای متد از فراخوانی آن در نخ جداگانه)، Monitor Object (همگام‌سازی دسترسی به متدهای یک شیء).

خلاصه اینکه: ایجاد = چطور بسازیم، ساختاری = چطور کنار هم بچینیم، رفتاری = چطور با هم حرف بزنیم.

۲۳ الگوی کلاسیک

Singleton-1

فقط یک نمونه از کلاس در کل برنامه بساز و یک نقطه دسترسی سراسری به آن بده. (مثال: مدیریت اتصال پایگاه داده)

Factory Method-2

یک متد برای ساخت شیء تعریف کن، ولی نوع دقیق شیء را به زیرکلاس‌ها واگذار کن. (مثال: ساخت دکمه‌های مختلف در سیستم‌عامل‌های گوناگون)

Abstract Factory-3

یک خانواده از اشیاء مرتبط را بدون مشخص کردن کلاس‌های عینی‌شان بساز. (مثال: ساخت دکمه + پنجره + منو برای یک تم خاص)

Builder-4

ساخت یک شیء پیچیده را از نمایش (نمایشگر) آن جدا کن تا بتوانی با یک فرایند، اشکال مختلف بسازی. (مثال: ساخت پیتزا با ترکیبات مختلف)

Prototype-5

یک شیء را با کپی‌کردن یک نمونه موجود (الگو) بساز، نه با new کردن. (مثال: کپی کردن یک سند ورد با تمام محتوا)

Adapter-6

رابط یک کلاس را به رابط دیگری که مشتری انتظار دارد، تبدیل کن (مثل مبدل برق). (مثال: تبدیل دوشاخه گرد به دو شاخه تخت)

Bridge-7

بعد انتزاعی را از بعد پیاده‌سازی جدا کن تا هر دو بتوانند مستقل تغییر کنند. (مثال: کنترل‌های از راه دور با پیاده‌سازی‌های مختلف برای تلویزیون و رادیو)

Composite-8

اشیاء را به صورت درخت بچین تا مشتری با یک شیء منفرد و یک ترکیب از اشیاء، یکسان رفتار کند. (مثال: فایل‌ها و پوشه‌ها در سیستم فایل)

Decorator-9

به یک شیء، قابلیت‌های جدید به صورت پویا اضافه کن بدون اینکه به ساختار آن آسیب بزنی. (مثال: قهوه ساده + شیر + شکلات + خامه)

Facade-10

یک رابط ساده‌تر برای یک زیرسیستم پیچیده فراهم کن. (مثال: دکمه "استارت" در ماشین که کلیه مراحل روشن کردن را پشت صحنه انجام می‌دهد)

Flyweight-11

با اشتراک‌گذاری اشیاء کوچک و سبک، حافظه را بهینه کن (مخصوص اشیاء زیاد). (مثال: حروف یک متن که بارها استفاده می‌شوند، هر کدام فقط یک بار ساخته می‌شوند)

Proxy-12

یک جانشین یا دروازه‌بان برای یک شیء دیگر قرار بده تا دسترسی به آن را کنترل کند. (مثال: تصویر کوچک (thumbnail) که جای تصویر سنگین را می‌گیرد تا بارگذاری نشده)

Strategy-13

یک خانواده از الگوریتم‌ها را تعریف کن و آنها را قابل تعویض در زمان اجرا کن. (مثال: پرداخت با کارت، پی‌پی، یا نقدی)

Observer-14

وقتی یک شیء تغییر می‌کند، به همه اشیاء وابسته به آن اطلاع بده. (مثال: خبرگزاری که به همه مشترکین خبر می‌دهد)

Command-15

درخواست را به صورت یک شیء بسته‌بندی کن تا قابلیت صف‌بندی، ثبت تاریخچه (Undo/Redo) داشته باشی. (مثال: دکمه‌های Ctrl+C و Ctrl+V که هر کدام یک فرمان هستند)

Iterator-16

به ترتیب به تمام عناصر یک مجموعه دسترسی پیدا کن بدون اینکه به ساختار داخلی آن دست بزنی. (مثال: حلقه for-each که آرایه یا لیست را پیمایش می‌کند)

Mediator-17

ارتباط بین چندین شیء را با یک واسطه مرکزی مدیریت کن تا اشیاء مستقیماً به هم متصل نباشند. (مثال: برج مراقبت که بین هواپیماها ارتباط برقرار می‌کند)

Memento-18

حالت داخلی یک شیء را ذخیره کن تا بتوانی بعداً آن را به حالت قبلی برگردانی. (مثال: دکمه Undo در ویرایشگر متن که به نقاط ذخیره‌شده برمی‌گردد)

State-19

رفتار یک شیء را با تغییر وضعیت داخلی‌اش عوض کن (انگار که کلاسش عوض شده). (مثال: پخش‌کننده موسیقی در حالت Play، Pause و Stop رفتار متفاوتی دارد)

Template Method-20

اسکلت یک الگوریتم را در متد پایه تعریف کن و بعضی قدم‌ها را به زیرکلاس‌ها واگذار کن. (مثال: دستور پخت غذا که مراحل ثابت است ولی مواد اولیه در هر غذا متفاوت است)

Visitor-21

یک عملیات جدید به یک ساختار از اشیاء اضافه کن بدون اینکه کلاس‌های آن اشیاء را تغییر دهی. (مثال: افزودن عملیات "ارسال ایمیل" به همه کاربران بدون تغییر کلاس User)

Chain of Responsibility-22

درخواست را از یک زنجیره از پردازشگرها عبور بده تا هر کدام توانست آن را پردازش کند. (مثال: سیستم پشتیبانی که درخواست از سطح ۱ به سطح ۲ و سطح ۳ می‌رود تا حل شود)

1. Creational Patterns (الگوهای ساخت اشیاء)

Singleton 1.1

• خلاصه کاربرد: وقتی فقط «یکی» از یه چیز باید وجود داشته باشه

مثل:

- 1 GameManager
- 2 ConfigManager
- 3 InputManager

و نمی‌خواهی چندتا ازش ساخته بشه.

این جا:

• می‌رسیم به Singleton

• (البته باید با دقت و فقط وقتی واقعاً لازمه استفاده بشه)

مشکل:

• فقط باید یک نمونه از یک کلاس در کل برنامه وجود داشته باشد.

• مثال: GameManager, Logger, GameConfig.

ایده پترن:

• سازنده را private می‌کنیم.

• یک نمونه‌ی ثابت (static) از کلاس داخل خودش نگه می‌داریم.

• یک متد getInstance() برای دسترسی به آن می‌نویسیم.

کد ساده:

```

1 public class GameConfig {
2     private static GameConfig instance;
3     private GameConfig() {
4         // تنظیمات اولیه
5     }
6     public static GameConfig getInstance() {
7         if (instance == null) {
8             instance = new GameConfig();
9         }
10        return instance;
11    }
12}

```

مزایا:

• کنترل کامل روی تعداد نمونه‌ها (فقط یکی).

• مناسب برای مدیریت تنظیمات سراسری.

معایب:

• اگر زیاد استفاده شود، تبدیل به Global Variable می‌شود.

• تست‌نویسی (Unit Test) را سخت می‌کند.

• وابستگی پنهان در سیستم ایجاد می‌کند.

مثال کامل: انتخاب کاپیتان

```

1 // Singleton
2 public final class Captain {
3     private static Captain captain;
4
5     private Captain() {
6
7     }
8
9     public static synchronized Captain getInstance(){
10         if (captain == null){
11             captain = new Captain();
12             System.out.println("New Captain is elected for your team");
13         }
14         else {
15             System.out.println("You already have a Captain for your team");
16             System.out.println("Send him for the toss");
17         }
18     }
19 }

```



```
18
19     return captain;
20 }
21 }
22
23 // Client
24 public class App
25 {
26     public static void main( String[] args )
27     {
28         Captain captain1 = Captain.getInstance();
29
30         Captain captain2 = Captain.getInstance();
31
32         if (captain1 == captain2) {
33             System.out.println("Captains are equal");
34         }
35     }
36 }
```

Factory Method 1.2

• خلاصه کاربرد: جلوگیری از تکرار new کردن کلاس‌های مشخص (مثلاً ساخت دشمن‌ها).

مشکل:

• کد شما نباید مستقیم با new کردن کلاس‌های مشخص، درگیر جزئیات ساخت شود.

• می‌خواهید بسته به شرایط، نوع مشخصی از یک آبجکت ساخته شود.

ایده پترن:

• یک متد کارخانه (factory) می‌سازید که شیء مناسب را برمی‌گرداند.

• کد کلاینت فقط آن متد را صدا می‌زند، نه `()new SomeClass`.

کد ساده:

```

1 abstract class Enemy {
2     abstract void attack();
3 }
4
5 class Orc extends Enemy {
6     @Override
7     void attack() { System.out.println("Orc attacks!"); }
8 }
9
10 class Troll extends Enemy {
11     @Override
12     void attack() { System.out.println("Troll attacks!"); }
13 }
1
2 class EnemyFactory {
3     public static Enemy createEnemy(String type) {
4         switch (type) {
5             case "orc": return new Orc();
6             case "troll": return new Troll();
7             default: throw new IllegalArgumentException("Unknown type");
8         }

```

```
9 }
10 }
```

مزایا:

- کد اصلی از جزئیات ساخت اشیاء جدا می‌شود.

- افزودن نوع جدید را ساده‌تر می‌کند (با کمی تغییر در کارخانه).

معایب:

- اگر شرط‌ها زیاد شوند، خود کارخانه می‌تواند شلوغ شود.

- هنوز هم برای هر نوع جدید باید کارخانه را تغییر دهید (نقض جزئی Open/Closed).

مثال کامل: رابط حمل و نقل

```
1 // Product
2 public interface Transport {
3     void deliver();
4 }
5
6 // Concrete Products
7 public class Truck implements Transport {
8     @Override
9     public void deliver() {
10        System.out.println("Delivering by truck...");
11    }
12 }
13
14 public class Ship implements Transport {
15     @Override
```

10-Java-Design-Patterns

```
16  public void deliver() {
17      System.out.println("Delivering by ship...");
18  }
19 }
20
21 public class Airplane implements Transport {
22     @Override
23     public void deliver() {
24         System.out.println("Delivering by airplane...");
25     }
26 }
27
28 // Creator
29 public abstract class Logistics {
30     protected abstract Transport createTransport();
31
32     //core business logic
33     public void planDelivery(){
34         Transport transport = createTransport();
35         transport.deliver();
36     }
37 }
38
39 // Concrete Creators
40 public class RoadLogistics extends Logistics {
41     @Override
42     protected Transport createTransport() {
```

10-Java-Design-Patterns

```
43     return new Truck();
44 }
45 }
46
47 public class SeaLogistics extends Logistics {
48     @Override
49     protected Transport createTransport() {
50         return new Ship();
51     }
52 }
53
54 public class SkyLogistics extends Logistics {
55     @Override
56     protected Transport createTransport() {
57         return new Airplane();
58     }
59 }
60
61 // Client
62 public class Main {
63     public static void main(String[] args) {
64         Logistics logistics;
65
66         //Road transportation
67         logistics = new RoadLogistics();
68
69         logistics.planDelivery();
70 }
```

```

71    //Sea transportation
72    logistics= new SeaLogistics();
73        logistics.planDelivery();
74
75        //sky logistics
76    logistics = new SkyLogistics();
77    logistics.planDelivery();
78 }
79 }

```

Abstract Factory 1.3

• خلاصه کاربرد: ساخت خانواده‌ای از اشیاء مرتبط (مثلاً ست کامل UI ویندوز در مقابل مک).

مشکل:

• می‌خواهید خانواده‌ای از اشیاء مرتبط بسازید که با هم کار می‌کنند.

• مثلاً: GUI برای Windows و Mac؛ هر کدام Button, Checkbox مخصوص خود را دارند.

ایده پترن:

• یک اینترفیس کارخانه کلی می‌سازید.

• هر پیاده‌سازی کارخانه، یک «خانواده کامل» از اشیاء را تولید می‌کند.

کد ساده:

```

1 interface GUIFactory {
2     Button createButton();
3     Checkbox createCheckbox();

```

```

4 }
5
6 interface Button { void render(); }
7 interface Checkbox { void render(); }
8
9 class WindowsButton implements Button {
10     public void render() { System.out.println("Windows Button"); }
11 }
12
13 class MacButton implements Button {
14     public void render() { System.out.println("Mac Button"); }
15 }
16
17 class WindowsFactory implements GUIFactory {
18     public Button createButton() { return new WindowsButton(); }
19     public Checkbox createCheckbox() { return () -> System.out.println("Windows Checkbox"); }
20 }
21
22 class MacFactory implements GUIFactory {
23     public Button createButton() { return new MacButton(); }
24     public Checkbox createCheckbox() { return () -> System.out.println("Mac Checkbox"); }
25 }

```

مزایا:

• تضمین می‌کند اشیاء مربوط به هم، هماهنگ ساخته شوند.

• وابستگی به اینترفیس‌ها، نه پیاده‌سازی‌ها.

معایب:

• برای پروژه‌های کوچک، اضافه‌کاری محسوب می‌شود.

• ساختار کلاس‌ها زیاد می‌شود.

Builder 1.4

• خلاصه کاربرد: ساخت کلاس‌های پیچیده با تنظیمات زیاد (مثل ساخت Character بازی).

وقتی می‌خوای شیء را بسازی، ولی سازنده‌ات شاخ‌دار شده

مثلاً:

```
1 new Character("Player1", 100, 20, 5, true, false, true, 0.2, "RANGER", ... );
```

این‌جا:

• خواندن و فهمیدن این سازنده سخت شده

• پارامتر زیاد = اشتباه انسانی زیاد

مشکل:

• سازنده‌ی کلاس بسیار طولانی و پر از پارامتر است.

• بعضی پارامترها اختیاری‌اند و خواندن کد سخت می‌شود.

ایده پترن:

• یک کلاس Builder می‌سازید که مرحله‌به‌مرحله پارامترها را تنظیم می‌کند.

• در نهایت با build() شیء آماده را برمی‌گرداند.

کد ساده:


```
1 class Character {
2     private String name;
3     private int hp;
4     private int damage;
5
6     private Character(Builder builder) {
7         this.name = builder.name;
8         this.hp = builder.hp;
9         this.damage = builder.damage;
10    }
11
12    public static class Builder {
13        private String name;
14        private int hp;
15        private int damage;
16
17        public Builder name(String name) {
18            this.name = name; return this;
19        }
20        public Builder hp(int hp) {
21            this.hp = hp; return this;
22        }
23        public Builder damage(int dmg) {
24            this.damage = dmg; return this;
25        }
26    }
27 }
```

```

12     public Character build() {
13         return new Character(this);
14     }
15 }
16 }
17
18 // استفاده:
19 Character hero = new Character.Builder()
20     .name("Hero")
21     .hp(100)
22     .damage(20)
23     .build();

```

مزایا:

- خوانایی بالا، مخصوصاً برای اشیاء پیچیده.

- جلوگیری از سازنده‌های طولانی.

معایب:

- کلاس اضافی (Builder) نیاز دارد.

- برای اشیاء ساده، ممکن است غیرضروری باشد.

مثال کامل: ساخت خانه

```

1 // Product
2 public class House implements HousePlan {
3
4     private String basement;

```

10-Java-Design-Patterns

```
5  private String structure;
6  private String roof;
7  private String interior;
8
9  @Override
10 public void setBasement(String basement) {
11     this.basement = basement;
12 }
13
14 @Override
15 public void setStructure(String structure) {
16     this.structure = structure;
17 }
18
19 @Override
20 public void setRoof(String roof) {
21     this.roof = roof;
22 }
23
24 @Override
25 public void setInterior(String interior) {
26     this.interior = interior;
27 }
28
29
30 @Override
31 public String toString() {
```

10-Java-Design-Patterns

```
32     return "House{" +
33         "basement=" + basement + "\" +
34         ", structure=" + structure + "\" +
35             ", roof='" + roof + '\"' +
36         ", interior=" + interior + "\" +
37         '}'';
38 }
39 }
40
41 // Builder Interface
42 public interface HouseBuilder {
43     void buildBasement();
44     void buildStructure();
45     void buildRoof();
46     void buildInterior();
47
48     House getHouse();
49 }
50
51 // Concrete Builders
52 public class IglooHouseBuilder implements HouseBuilder {
53     private House house;
54
55     public IglooHouseBuilder() {
56         this.house = new House();
57     }
58 }
```

```
59  @Override
60  public void buildBasement() {
61      house.setBasement("ice");
62  }
64  @Override
65  public void buildStructure() {
66      house.setStructure("ice");
67  }
68
69  @Override
70  public void buildRoof() {
71      house.setRoof("ice");
72  }
73
74  @Override
75  public void buildInterior() {
76      house.setInterior("ice");
77  }
78
79  @Override
80  public House getHouse() {
81      return house;
82  }
83 }
84
85 public class TipiHouseBuilder implements HouseBuilder {
86     private House house;
87
88     public TipiHouseBuilder() {
```

10-Java-Design-Patterns

```
89     this.house = new House();
90 }
91
92 @Override
93 public void buildBasement() {
94     house.setBasement("wood");
95 }
96
97 @Override
98 public void buildStructure() {
99     house.setStructure("wood");
100 }
101
102 @Override
103 public void buildRoof() {
104     house.setRoof("wood");
105 }
106
107 @Override
108 public void buildInterior() {
109     house.setInterior("wood");
110 }
111
112 @Override
113 public House getHouse() {
114     return house;
115 }
```

10-Java-Design-Patterns

```
116 }
117
118 // Director
119 public class CivilEngineer {
120     private HouseBuilder houseBuilder;
121
122     public CivilEngineer(HouseBuilder houseBuilder) {
123         this.houseBuilder = houseBuilder;
124     }
125
126     public House getHouse() {
127         return this.houseBuilder.getHouse();
128     }
129
130     public void constructHouse() {
131         this.houseBuilder.buildBasement();
132         this.houseBuilder.buildStructure();
133         this.houseBuilder.buildRoof();
134         this.houseBuilder.buildInterior();
135     }
136 }
137
138 // Main
139 public class Main {
140     public static void main(String[] args) {
141         HouseBuilder iglooBuilder = new IglooHouseBuilder();
142         CivilEngineer engineer = new CivilEngineer(iglooBuilder);
```

```
143
144     engineer.constructHouse();
145
146     House house = engineer.getHouse();
147
148     System.out.println(house);
149
150     StringBuilder builder = new StringBuilder("Hello");
151
152         String data = builder.append(1)
153             .append(true)
154             .append("friend")
155         .toString();
156
157         System.out.println(data);
158
159     }
160 }
```

Prototype 1.5

• خلاصه کاربرد: ساخت اشیاء جدید از روی نمونه‌های آماده، وقتی new کردن هزینه‌بر است.

مشکل:

• ساخت یک شیء جدید، هزینه‌بر است (از نظر زمان یا منابع).

• ولی یک شیء مشابه در اختیار دارید و می‌خواهید از روی آن کپی بسازید.

ایده پترن:

• به جای `new`, از `clone()` یا کپی کردن شیء موجود استفاده می‌کنید.

کد ساده:

```

1 class Enemy implements Cloneable {
2     private String type;
3     private int hp;
4
5     public Enemy(String type, int hp) {
6         this.type = type;
7         this.hp = hp;
8     }
9
10    @Override
11    public Enemy clone() {
12        try {
13            return (Enemy) super.clone();
14        } catch (CloneNotSupportedException e) {
15            throw new RuntimeException(e);
16        }
17    }
18 }
19
20 // استفاده:
21 Enemy orc = new Enemy("orc", 100);
22 Enemy clonedOrc = orc.clone();

```

مزایا:

- ایجاد اشیاء جدید سریع‌تر از ساخت از صفر.
- مناسب برای سیستم‌هایی مانند بازی که دشمن‌های مشابه زیاد دارد.

معایب:

- پیاده‌سازی clone() در جاوا کمی حساس و پیچیده است.
- برای اشیاء دارای وابستگی عمیق (Deep Copy)، دقت زیادی لازم است.

2. Structural Patterns (الگوهای ساختاری)

Adapter 2.1

• خلاصه کاربرد: اتصال دو کلاس با اینترفیس‌های ناسازگار بدون تغییر کد اصلی.

مشکل:

- دو کلاس/کتابخانه با هم ناسازگار هستند (متدها یا نام‌ها متفاوتند).
- می‌خواهید بدون تغییر کد اصلی، از آن‌ها با هم استفاده کنید.

ایده پترن:

• یک «آداپتر» می‌نویسید که رابط (interface) مورد انتظار شما را پیاده‌سازی کرده و پشت‌صحنه، کلاس خارجی را صدا بزند.

کد ساده:

```

1  interface GameInput {
2      void jump();
3  }
4
5  // کتابخانه خارجی:
6  class ExternalInputSystem {
7      void doJump() {
8          System.out.println("External jump");
9      }
10 }
11
12 class InputAdapter implements GameInput {
13     private ExternalInputSystem external;
14
15     public InputAdapter(ExternalInputSystem external) {
16         this.external = external;
17     }
18
19     @Override
20     public void jump() {
21         external.doJump();
22     }
23 }

```

مزایا:

• اتصال کد قدیمی به کتابخانه‌های جدید بدون تغییر هسته.

• جداسازی وابستگی‌ها.

معایب:

• اگر زیاد استفاده شود، تعداد کلاس‌ها زیاد می‌شود.

• می‌تواند ساختار را پیچیده کند.

Bridge 2.2

• خلاصهٔ کاربرد: جدا کردن انتزاع از پیاده‌سازی وقتی کلاس دو بُعد تغییر دارد (مثل شکل و نوع رندر).

مشکل:

• یک کلاس در دو بُعد تغییر می‌کند (مثلاً نوع سلاح و نوع رندر).

• اگر برای هر ترکیب یک کلاس بنویسی، تعداد کلاس‌ها منفجر می‌شود.

ایده‌ای پترن:

• یک کلاس را به دو بخش: «Abstraction» و «Implementation» تقسیم می‌کنی.

• این دو از طریق ترکیب (composition) به هم وصل می‌شوند، نه ارث‌بری.

کد ساده:

```
1 interface Renderer {
2     void renderCircle(float x, float y, float radius);
3 }
4
5 class VectorRenderer implements Renderer {
6     public void renderCircle(float x, float y, float radius) {
7         System.out.println("Drawing circle as vector");
8     }
9 }
```

```

10
11 class RasterRenderer implements Renderer {
12     public void renderCircle(float x, float y, float radius) {
13         System.out.println("Drawing circle as pixels");
14     }
15 }
16
17 class Circle {
18     private Renderer renderer;
19     private float x, y, radius;
20
21     public Circle(Renderer renderer, float x, float y, float radius) {
22         this.renderer = renderer;
23         this.x = x;
24         this.y = y;
25         this.radius = radius;
26     }
27
28     public void draw() {
29         renderer.renderCircle(x, y, radius);
30     }
31 }

```

مزایا:

• کاهش انفجار تعداد کلاس‌ها.

• امکان تغییر مستقل دو بُعد (مثلاً نوع رندر و نوع شکل).

معایب:

• ساختار اولیه کمی پیچیده‌تر است.

Composite 2.3

• خلاصه کاربرد: مدیریت ساختارهای درختی (مثل صحنه بازی: Node, Child, Group) با یک API یکسان.

مشکل:

• می‌خواهید با ساختارهای درختی (مثل صحنه‌ی بازی: Node, Child Node) کار کنید.

• می‌خواهید "یک شیء" و "مجموعه‌ای از اشیاء" را با یک API مدیریت کنید.

ایده پترن:

• یک اینترفیس مشترک تعریف می‌کنید.

• هم اشیاء منفرد و هم مجموعه‌ها آن را پیاده‌سازی می‌کنند.

کد ساده:

```
1 interface GameObject {
2     void update();
3 }
4
5 class Sprite implements GameObject {
6     public void update() {
7         System.out.println("Sprite update");
8     }
9 }
10
```

```

11 class Group implements GameObject {
12     private List<GameObject> children = new ArrayList<>();
13
14     public void add(GameObject obj) { children.add(obj); }
15
16     public void update() {
17         for (GameObject obj : children) {
18             obj.update();
19         }
20     }
21 }

```

مزایا:

• یکسان شدن کار با تکی و مجموعه.

• مناسب برای ساختارهای صحنه (Scene Graph).

معایب:

• اشکال زدایی ساختارهای پیچیده ممکن است سخت شود.

Decorator 2.4

• خلاصه کاربرد: افزودن قابلیت‌های جدید به شیء بدون تغییر کلاس اصلی (مانند شمشیر آتشین).

مشکل:

• می‌خواهید به یک شیء، رفتار جدید اضافه کنید بدون اینکه کلاس اصلی را تغییر دهید.

• مثلاً روی یک سلاح، قابلیت «آتش» یا «یخ» اضافه کنید.

ایده پترن:

• کلاسی می‌سازید که اینترفیس شیء اصلی را پیاده‌سازی کرده و یک شیء از همان نوع را در خود نگه دارد؛ سپس قبل/بعد از صدا زدن متدهای آن، رفتار اضافه می‌کند.

کد ساده:

```
1 interface Weapon {
2     void attack();
3 }
4
5 class BasicSword implements Weapon {
6     public void attack() {
7         System.out.println("Slash");
8     }
9 }
1
12 class FireDecorator implements Weapon {
13     private Weapon base;
14
15     public FireDecorator(Weapon base) {
16         this.base = base;
17     }
18
19     public void attack() {
20         base.attack();
21         System.out.println("+ Fire damage");
22     }
23 }
```



```
12 }
```

```
13 }
```

مزایا:

• افزودن رفتار بدون تغییر کلاس اصلی.

• ترکیب پذیری بالا (مثلاً Fire + Ice + Poison).

معایب:

• ممکن است زنجیره‌ی دکوراتورها پیچیده شود.

• ردیابی رفتار واقعی سخت می‌شود.

Facade 2.5

• خلاصه کاربردی: ایجاد یک رابط ساده برای یک سیستم پیچیده و شلوغ.

مشکل:

• سیستم شما از چندین کلاس و زیرسیستم تشکیل شده و استفاده مستقیم از آن‌ها سخت است.

ایده پترن:

• یک کلاس ساده (Facade) می‌سازید که متدهای ساده و سطح بالا ارائه می‌کند و پشت صحنه بقیه کلاس‌ها را صدا

می‌زند.

کد ساده:

```
1 class AudioSystem {
2     void init() {}
3     void playSound(String name) {}
```

```
4 }  
5  
6 class PhysicsSystem {  
7     void init() {}  
8 }  
9  
10 class GameEngineFacade {  
11     private AudioSystem audio = new AudioSystem();  
12     private PhysicsSystem physics = new PhysicsSystem();  
13  
14     public void startGame() {  
15         audio.init();  
16         physics.init();  
17         System.out.println("Game started");  
18     }  
19 }
```

مزایا:

• ساده سازی استفاده از سیستم های پیچیده.

• مناسب برای API های سطح بالا.

معایب:

• اگر بیش از حد بزرگ شود، خودش تبدیل به God Object می شود.

Flyweight 2.6

• خلاصه کاربرد: صرفه‌جویی در رم با اشتراک‌گذاری داده‌های مشترک بین میلیون‌ها شیء مشابه.

مشکل:

• تعداد بسیار زیادی از اشیاء مشابه دارید (مثلاً هزاران سرباز در یک بازی)، و هر کدام داده‌های تکراری زیادی دارند.

ایده پترن:

• داده‌های مشترک (invariant) را جدا کرده و فقط یک نسخه نگه می‌دارید.

• داده‌های خاص هر شیء را جداگانه ذخیره می‌کنید.

کد ساده (خیلی ساده‌شده):

```
1 class TreeType {
2     String name;
3     String texture;
4
5     public TreeType(String name, String texture) {
6         this.name = name;
7         this.texture = texture;
8     }
9 }
10
11 class Tree {
12     int x, y;
13     TreeType type;
14 }
```

مزایا:

• کاهش مصرف حافظه.

معایب:

• پیچیدگی در مدیریت داده‌های مشترک و اختصاصی.

Proxy 2.7

• خلاصه کاربرد: کنترل دسترسی یا مدیریت پیش‌از-اجرا (مانند Lazy Loading تصویر).

مشکل:

• می‌خواهید قبل از دسترسی به یک شیء، کنترل اضافه انجام دهید (مثلاً Lazy Loading، یا کنترل دسترسی، یا کش).

ایده پترن:

• یک کلاس Proxy می‌سازید که اینترفیس همان شیء اصلی را دارد، و درخواست‌ها را قبل از رسیدن به شیء اصلی مدیریت می‌کند.

کد ساده:

```
1 interface Image {
2     void display();
3 }
4
5 class RealImage implements Image {
6     private String filename;
7
8     public RealImage(String filename) {
9         this.filename = filename;
10        loadFromDisk();
11    }
```

```

12
13     private void loadFromDisk() {
14         System.out.println("Loading " + filename);
15     }
16
17     public void display() {
18         System.out.println("Displaying " + filename);
19     }
20 }
21
22 class ImageProxy implements Image {
23     private String filename;
24     private RealImage realImage;
25
26     public ImageProxy(String filename) {
27         this.filename = filename;
28     }
29
30     public void display() {
31         if (realImage == null) {
32             realImage = new RealImage(filename);
33         }
34         realImage.display();
35     }
36 }

```

مزایا:

• کنترل دسترسی، Lazy Loading، کش کردن.

معایب:

• افزودن یک لایه اضافی بین کلاپنت و شیء اصلی.

3. Behavioral Patterns (الگوهای رفتاری)

Strategy 3.1

• خلاصه کاربرد: جایگزینی if-else های طولانی برای رفتارهای مختلف (مثل الگوریتم های حرکت دشمن).

مثلاً:

```
1 if (type.equals("CREDIT_CARD")) { ... }
1 else if (type.equals("PAYPAL")) { ... }
2 else if (type.equals("CRYPTO")) { ... }
```

و هر جا، همین داستان تکرار می شه.

این نشونه ی اینه که:

• داری به نوع (type) حساسیت نشون می دی

• هر "نوع" رفتار متفاوتی داره

مشکل:

• چندین روش مختلف برای انجام یک کار دارید (مثلاً روش های مختلف پرداخت، یا روش های مختلف حرکت دشمن)، و نمی خواهید با if-else تشخیص دهید.

ایده پترن:

• یک اینترفیس برای رفتار تعریف می کنید.

• چندین پیاده سازی مختلف برایش می نویسید.

• در زمان اجرا، یکی از آن ها را به شیء اصلی تزریق می کنید.

```
1 interface PaymentStrategy {
2     void pay(double amount);
3 }
4
5 class CreditCardPayment implements PaymentStrategy {
6     public void pay(double amount) {
7         System.out.println("Paying " + amount + " by credit card");
8     }
9 }
10
11 class CryptoPayment implements PaymentStrategy {
12     public void pay(double amount) {
13         System.out.println("Paying " + amount + " by crypto");
14     }
15 }
16
17 class PaymentProcessor {
18     private PaymentStrategy strategy;
19
20     public PaymentProcessor(PaymentStrategy strategy) {
21         this.strategy = strategy;
22     }
23
24     public void process(double amount) {
25         strategy.pay(amount);
26     }
27 }
```

```
18 }
19 }
```

مزایا:

• جلوگیری از if-else های طولانی.

• امکان تغییر رفتار در زمان اجرا.

معایب:

• افزایش تعداد کلاس ها.

مثال کامل: مسیر حرکت

```
1 // Strategy interface
2 public interface RouteStrategy
3 {
4     void buildRoute(String from, String to);
5 }
6
7 // Navigator
8 public class Navigator
9 {
10     private RouteStrategy routeStrategy;
11
12     public void setRouteStrategy(RouteStrategy routeStrategy)
13     {
14         this.routeStrategy = routeStrategy;
15     }
16
17     // Wrapper (📌)
18     public void buildRoute(String from, String to)
```


10-Java-Design-Patterns

```
19     {
20         this.routeStrategy.buildRoute(from, to);
21     }
22 }
23
24 // Car Concerete
25 public class CarRouteStrategy implements RouteStrategy
26 {
27     @Override
28     public void buildRoute(String from, String to)
29     {
30         System.out.println("Driving...");
31     }
32 }
33
34 // Walking Concerete
35 public class WalkingRouteStrategy implements RouteStrategy
36 {
37     @Override
38     public void buildRoute(String from, String to)
39     {
40         System.out.println("Walking...");
41     }
42 }
43
44 // Main
45 public class Main
46 {
47     public static void main(String[] args)
48     {
49         RouteStrategy carStrategy = new CarRouteStrategy();
50
```

```

51     Navigator navigator = new Navigator();
52     navigator.setRouteStrategy(carStrategy);
53     navigator.buildRoute("Tehran", "Qazvin");
54 }
55 }

```

Observer 3.2

• خلاصه کاربرد: خبردار کردن سایر اشیاء از تغییرات (مانند آپدیت UI هنگام تغییر جان بازیکن).

مشکل:

• یک شیء (Subject) دارد تغییر می‌کند و چندین بخش دیگر باید از این تغییر باخبر شوند (مثلاً UI، لاگ، سیستم امتیاز).

ایده پترن:

• لیستی از ناظرها (Observers) در Subject نگه می‌دارید.

• وقتی Subject تغییر کرد، همه Observerها را خبر می‌کند.

کد ساده:

```

1 interface Observer {
2     void update(int hp);
3 }
4
5 class Player {
6     private int hp;
7     private List<Observer> observers = new ArrayList<>();
8

```

```

9  public void addObserver(Observer o) { observers.add(o); }
10
11  public void setHp(int hp) {
12      this.hp = hp;
13      notifyObservers();
14  }
15
16  private void notifyObservers() {
17      for (Observer o : observers) {
18          o.update(hp);
19      }
20  }
21 }

```

مزایا:

• Loose Coupling بین Subject و Observer ها.

معایب:

• ممکن است چرخه‌های پیچیده‌ای از اطلاع‌رسانی ایجاد شود.

Command 3.3

• خلاصه کاربرد: پیمایش روی کالکشن‌ها بدون درگیری با جزئیات داخلی آن‌ها.

مشکل:

• می‌خواهید درخواست (عملیاتی مثل "حمله"، "پريدن") را به شکل یک شیء ذخیره کنید (برای Undo، Redo، Queue کردن دستورات).

ایده پترن:

• هر دستور را در یک کلاس جداگانه می‌ریزید که یک متد execute() دارد.

کد ساده:

```
1 interface Command {
2     void execute();
3 }
4
5 class JumpCommand implements Command {
6     public void execute() {
7         System.out.println("Player jumps");
8     }
9 }
10
11 class InputHandler {
12     private Command jumpCommand;
13
14     public InputHandler(Command jumpCommand) {
15         this.jumpCommand = jumpCommand;
16     }
17
18     public void onJumpPressed() {
19         jumpCommand.execute();
20     }
21 }
```

مزایا:

• مناسب برای Undo/Redo و Queue از دستورات.

معایب:

• افزایش تعداد کلاس‌های کوچک.

مثال کامل: تلویزیون

```
1 // Command
2 public interface Command
3 {
4     void execute();
5 }
6
7 // Receiver
8 public class Television
9 {
10     public void turnOn()
11     {
12         System.out.println("TV is on.");
13     }
14
15     public void turnOff()
16     {
17         System.out.println("TV is off.");
18     }
19 }
20
21 // Concrete Command
22 public class TurnOnCommand implements Command
23 {
24     private Television television;
```

10-Java-Design-Patterns

```
25
26     public TurnOnCommand(Television television)
27     {
28         this.television = television;
29     }
30
31     // Wrappper
32     @Override
33     public void execute()
34     {
35         this.television.turnOn();
36     }
37 }
38
39 // Concerete Command
40 public class TurnOffCommand implements Command
41 {
42     private Television television;
43
44     public TurnOffCommand(Television television)
45     {
46         this.television = television;
47     }
48
49     @Override
50     public void execute()
51     {
52         this.television.turnOff();
53     }
54 }
55
```

10-Java-Design-Patterns

```
56 // Invoker
57 public class RemoteControllerInvoker implements Command
58 {
59     private Command command;
60
61     public void setCommand(Command command)
62     {
63         this.command = command;
64     }
65
66     @Override
67     public void execute()
68     {
69         this.command.execute();
70     }
71 }
72
73 // Main
74 public class Main
75 {
76     public static void main(String[] args)
77     {
78         Television television = new Television();
79
80         Command turnOn = new TurnOnCommand(television);
81         Command turnOff = new TurnOffCommand(television);
82         Command volumeUp = new VolumeUpCommand(television);
83         Command volumeDown = new VolumeDownCommand(television);
84
85         RemoteControllerInvoker remote = new RemoteControllerInvoker();
86         remote.setCommand(turnOn);
```

```
87         remote.execute();
88
89         remote.setCommand(turnOff);
90         remote.execute();
91
92         remote.setCommand(volumeUp);
93         remote.execute();
94
95         remote.setCommand(volumeDown);
96         remote.execute();
97     }
98 }
```

Iterator 3.4

• خلاصه کاربرد: پیمایش روی کالکشن‌ها بدون درگیری با جزئیات داخلی آن‌ها.

مشکل:

• می‌خواهید روی عناصر یک مجموعه حرکت کنید بدون اینکه جزئیات ساختار داخلی آن را بدانید.

ایده پترن:

• یک اینترفیس Iterator تعریف می‌کنید که hasNext() و next() داشته باشد.

کد:

در جاوا، Iterator از قبل وجود دارد (java.util.Iterator).


```
1 List<String> list = new ArrayList<>();
2 Iterator<String> it = list.iterator();
3 while (it.hasNext()) {
4     System.out.println(it.next());
5 }
```

مزایا:

- ساده‌سازی پیمایش روی ساختارهای مختلف.

معایب:

- تقریباً همیشه ضمنی در جاوا استفاده می‌شود.

Mediator 3.5

- خلاصه کاربرد: کاهش وابستگی‌های مستقیم بین کلاس‌ها با استفاده از یک واسط مرکزی.

مشکل:

- چندین کلاس با هم زیاد در ارتباط‌اند و هر کدام دیگری را صدا می‌زنند.

- این باعث وابستگی شدید و کد پیچیده می‌شود.

ایده پترن:n:

- به جای اینکه مستقیم همدیگر را صدا بزنند، از یک واسط (Mediator) استفاده می‌کنند.

کد ساده (مفهومی):

10-Java-Design-Patterns

```
1 interface Mediator {  
2     void notify(Component sender, String event);  
3 }  
4  
5 abstract class Component {  
6     protected Mediator mediator;  
7     public Component(Mediator mediator) { this.mediator = mediator; }  
8 }
```

مزایا:

• کاهش وابستگی مستقیم بین کلاس‌ها.

معایب:

• Mediator ممکن است خیلی بزرگ و پیچیده شود.

Memento 3.6

• خلاصه کاربرد: ذخیره وضعیت فعلی برای بازگشت به عقب (Save/Undo).

مشکل:

• می‌خواهید وضعیت یک شیء را ذخیره کنید تا بعداً به آن برگردید (Undo).

ایده پترن:

• یک کلاس Memento می‌سازید که وضعیت را نگه می‌دارد.

• شیء اصلی می‌تواند وضعیتش را به Memento تبدیل و برگردان کند.

کد ساده:

10-Java-Design-Patterns

```
1  class GameState {
2      private int level;
3
4      public GameState(int level) {
5          this.level = level;
6      }
7
8      public int getLevel() { return level; }
9  }
10
11 class Game {
12     private int level;
13
14     public GameState save() {
15         return new GameState(level);
16     }
17
18     public void restore(GameState state) {
19         this.level = state.getLevel();
20     }
21 }
```

مزایا:

• پیاده‌سازی Undo/Save در بازی‌ها.

معایب:

• مصرف حافظه زیاد اگر وضعیت‌ها بزرگ باشند.

State 3.7

• خلاصه کاربرد: مدیریت رفتار شیء بر اساس وضعیتش (مثل Standing, Attacking) به جای if.

مشکل:

• رفتار یک شیء بسته به وضعیت (State) آن تغییر می‌کند.

• کد پر از if (state == ...) می‌شود.

ایده پترن:

• برای هر وضعیت، یک کلاس جداگانه می‌سازید که رفتار را تعریف می‌کند.

کد ساده:

```
1 interface PlayerState {
2     void handleInput();
3 }
4
5 class StandingState implements PlayerState {
6     public void handleInput() {
7         System.out.println("Standing");
8     }
9 }
10
11 class RunningState implements PlayerState {
12     public void handleInput() {
13         System.out.println("Running");
14     }
15 }
16
17 class Player {
```

10-Java-Design-Patterns

```
18     private PlayerState state;
19
20     public void setState(PlayerState state) {
21         this.state = state;
22     }
23
24     public void handleInput() {
25         state.handleInput();
26     }
27 }
```

مزایا:

• حذف if-else های مربوط به وضعیت.

*معایب:

• افزایش تعداد کلاس ها.

مثال کامل: ثبت مقاله در سایت

```
1 // State
2 public interface DocumentState
3 {
4     void publish();
5
6     void render();
7
8     void approve();
9 }
10
```

10-Java-Design-Patterns

```
11 // Document
12 public class Document
13 {
14     private DocumentState documentState;
15
16     public Document()
17     {
18         // Always Draft for first
19         this.documentState = new DraftState(this);
20     }
21
22     public void changeState(DocumentState state)
23     {
24         this.documentState = state;
25     }
26
27     public void publish()
28     {
29         this.documentState.publish();
30     }
31
32     public void render()
33     {
34         this.documentState.render();
35     }
36
37     public void approve()
38     {
39         this.documentState.approve();
40     }
41 }
```

```
42
43 1 // Draft
44 1 public class DraftState implements DocumentState
45 1 {
46     private Document document;
47
48     public DraftState(Document document)
49     {
50         this.document = document;
51     }
52
53     @Override
54     public void approve()
55     {
56         System.out.println("Draft can not be Approved...");
57     }
58
59     @Override
60     public void publish()
61     {
62         System.out.println("Document is send moderation...");
63         this.document.changeState(new ModerationState(document));
64     }
65
66     @Override
67     public void render()
68     {
69         System.out.println("Document Render...");
70     }
71 1 }
72
```

10-Java-Design-Patterns

```
73 1 // Moderation
74 1 public class ModerationState implements DocumentState
75 1 {
76     private Document document;
77
78     public ModerationState(Document document)
79     {
80         this.document = document;
81     }
82
83     @Override
84     public void approve()
85     {
86         System.out.println("Document approve and publish...");
87         this.document.changeState(new PublishState(document));
88     }
89
90     @Override
91     public void publish()
92     {
93         System.out.println("Document is waiting for approval...");
94     }
95
96     @Override
97     public void render()
98     {
99         System.out.println("Rendering document under moderation state...");
100     }
101 1 }
102
```


10-Java-Design-Patterns

```
103 1 // Publish
104 1 public class PublishState implements DocumentState
105 1 {
106     private Document document;
107
108     public PublishState(Document document)
109     {
110         this.document = document;
111     }
112
113     @Override
114     public void approve()
115     {
116         System.out.println("Document is already approced");
117     }
118
119     @Override
120     public void publish()
121     {
122         System.out.println("Document is already published");
123     }
124
125     @Override
126     public void render()
127     {
128         System.out.println("Rendering published document...");
129     }
130 1 }
131
```

```

132 1 // Main
133 1 public class Main
134 1 {
135     public static void main(String[] args)
136     {
137         Document document = new Document();
138
139         // Draft
140         document.render();
141         document.publish();
142
143         // Moderation
144         document.render();
145         document.publish();
146         document.approve();
147
148         // Publish
149         document.render();
150         document.publish();
151     }
152 1 }

```

Template Method 3.8

• خلاصه کاربرد: تعریف اسکلت کلی الگوریتم در کلاس پایه و سپردن جزئیات به فرزندان.

مشکل:

• الگوریتم کلی در چند کلاس مشابه است، اما بعضی جزئیات فرق می‌کند.

ایده پترن:

• در کلاس پایه، الگوریتم کلی را به صورت متد final می‌نویسید.

• بخش‌هایی که باید متفاوت باشند را به صورت متدهای abstract یا قابل override تعریف می‌کنید.

کد ساده:

```
1 abstract class GameLoop {
2     public final void run() {
3         init();
4         update();
5         render();
6     }
7
8     protected abstract void init();
9     protected abstract void update();
10    protected abstract void render();
11 }
```

مزایا:

• اشتراک الگوریتم کلی، جلوگیری از تکرار کد.

معایب:

• وابستگی به ارث‌بری (Inheritance).

Visitor 3.9

• خلاصه کاربرد: افزودن عملیات جدید به کلاس‌ها بدون تغییر ساختار داخلی آن‌ها.

مشکل:

• می‌خواهید روی مجموعه‌ای از کلاس‌های مختلف عملیاتی انجام دهید بدون اینکه کلاس‌های اصلی را تغییر دهید.

ایده پترن:

• یک Visitor تعریف می‌کنید که متد visit برای هر نوع کلاس دارد.

• کلاس‌ها یک متد accept(Visitor v) دارند.

کد ساده (خیلی خلاصه):

```
1 interface Visitor {  
2     void visitEnemy(Enemy e);  
3     void visitPlayer(Player p);  
4 }  
5  
6 interface Element {  
7     void accept(Visitor v);  
8 }
```

مزایا:

• افزودن عملیات جدید بدون تغییر کلاس‌های موجود.

معایب:

• اگر انواع جدیدی از Element اضافه کنید، باید Visitor ها را تغییر دهید.

Chain of Responsibility 3.10

• خلاصه کاربرد: عبور درخواست از زنجیره‌ای از هندلرها تا یکی مسئولیت آن را بپذیرد.

مشکل:

• می‌خواهید چندین Handler داشته باشید که هر کدام ممکن است یک درخواست را پردازش کنند.

• نمی‌خواهید یک Handler خاص به صورت ثابت مشخص باشد.

ایده پترن:

• Handlerها را در یک زنجیره قرار می‌دهید.

• هر Handler درخواست را گرفته، اگر نتوانست رسیدگی کند، به بعدی پاس می‌دهد.

کد ساده:

```
1 abstract class Handler {  
2     protected Handler next;  
3  
4     public Handler setNext(Handler next) {  
5         this.next = next;  
6         return next;  
7     }  
8  
9     public void handle(String request) {  
10        if (next != null) next.handle(request);  
11    }  
12 }
```

```

13
14 class AuthHandler extends Handler {
15     @Override
16     public void handle(String request) {
17         System.out.println("Auth checked");
18         super.handle(request);
19     }
20 }

```

مزایا:

• انعطاف بالا در تعیین ترتیب و نوع Handler ها.

معایب:

• دیباگ کردن مسیر عبور درخواست گاهی سخت است.

نکته پایانی برای فالكون

• از اسم پترن شروع نکن؛ از مشکل شروع کن.

• هر وقت دیدی:

• if-else بر اساس نوع زیاد شده → به Strategy / State فکر کن.

• ساخت اشیاء پیچیده است → به Builder / Factory فکر کن.

• رفتار مشترک با تفاوت‌های کوچک داری → به Template Method فکر کن.

• چندین آبجکت باید از تغییر یک آبجکت باخبر شوند → به Observer فکر کن.

این فایل را می‌توانی به عنوان design-pattern-cheatsheet.md در GitHub یا Gist ذخیره کنی و هر وقت وسط پروژه گیر کردی، سریع نگاهی به آن بیندازی.